

Módulo 3: Cierre del Curso

Cómo usar bien la IA: contexto, flujos y artefactos

2026-04-30

Enlaces

-  segana@fen.uchile.cl
 -  <https://segana.netlify.app>
 -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
 -  <https://github.com/sebaegana>
-

Módulo 03

Clase de Cierre

Cómo usar bien la IA: contexto, flujos y artefactos

Objetivo de la sesión

- Entender por qué el contexto es el insumo más crítico de un sistema de IA
 - Revisar las principales formas de generar y entregar contexto
 - Ver cómo un flujo multi-paso produce artefactos que alimentan etapas siguientes
 - Conectar estas ideas con las capacidades vistas en el módulo
-

Estructura de esta sesión

1. el papel del contexto
 2. formas de generar contexto
 3. spec-driven development y su relación con los artefactos
 4. flujo multi-paso con artefactos estandarizados
 5. actividad de cierre
-

Parte I

El papel del contexto

El modelo no sabe nada de tu organización

Un LLM por sí solo:

- no conoce tu industria
- no sabe qué proceso quieres automatizar
- no tiene acceso a tus documentos internos
- no recuerda lo que conversaste ayer
- no sabe cuáles son las reglas de tu negocio

Consecuencia directa

La calidad de la respuesta depende casi completamente de la calidad del contexto que le entregás.

¿Qué es el contexto?

Es todo lo que rodea la pregunta del usuario y le da sentido al modelo.

Incluye:

- instrucciones de comportamiento
 - documentos de referencia
 - historial de conversación
 - datos de la sesión
 - herramientas disponibles
 - ejemplos de entradas y salidas esperadas
 - reglas del negocio o del dominio
-

La regla más importante

Basura entra, basura sale.

Un modelo potente con contexto pobre produce salidas pobres.

Un modelo más modesto con contexto bien construido puede producir salidas mucho más útiles.

Traducción práctica

El trabajo real de usar bien la IA no está en elegir el modelo más grande, sino en construir bien el contexto.

Tres tipos de contexto según su origen

Tipo	De dónde viene	Ejemplo
Estático	se define una vez y persiste	instrucciones del sistema, reglas del negocio
Recuperado	se busca en el momento según la consulta	documentos, FAQ, normativa vía RAG
Generado	se produce durante el flujo y alimenta etapas siguientes	output de una etapa anterior

Por qué el contexto generado es el más valioso

En flujos con múltiples pasos, cada etapa puede producir un artefacto que se convierte en contexto de la siguiente.

Esto permite:

- encadenar análisis progresivos
- reducir lo que el modelo tiene que inferir
- hacer el proceso más trazable y auditable
- reutilizar resultados intermedios

Idea clave

El flujo bien diseñado convierte el trabajo previo en contexto explícito y controlado.

Parte II

Formas de generar contexto

Panorama general

Hay varias formas de entregar o construir contexto para un modelo:

1. instrucciones del sistema (CLAUDE.md, system prompt)
2. skills
3. planes
4. MCP (Model Context Protocol)
5. RAG
6. memoria persistente
7. ejemplos (few-shot)

Cada una tiene un rol distinto y un momento de uso distinto.

1. Instrucciones del sistema

Es el contexto estático más fundamental.

Qué es

Un bloque de texto que describe al modelo cómo debe comportarse, qué sabe y qué no debe hacer.

Dónde vive

- en el `system prompt` de la API
- en un archivo `CLAUDE.md` para proyectos con Claude Code
- en la configuración de un chatbot o asistente

Ejemplo

```
Eres un asistente interno de la AFP Modelo.  
Solo respondes preguntas sobre pensiones usando la normativa oficial.  
No estimas montos ni das recomendaciones de inversión.  
Si no tienes información suficiente, derivas al equipo de atención.
```

CLAUDE.md como instrucción persistente de proyecto

En Claude Code, el archivo `CLAUDE.md` cumple el rol de instrucción del sistema a nivel de proyecto.

Qué suele incluir

- descripción del proyecto y su propósito
- reglas de estilo de código
- comandos útiles para ese repositorio
- restricciones y convenciones del equipo
- contexto relevante para el agente

Por qué importa

Cada vez que el agente abre el proyecto, lee ese archivo. No hay que repetir el contexto en cada conversación.

Ejemplo de CLAUDE.md

```
# Proyecto: Asistente de análisis regulatorio - AFP Modelo

## Descripción
Este proyecto construye un asistente interno para analistas del área de cumplimiento.
Responde preguntas sobre normativa previsional usando documentos oficiales de la CMF y la SFR.

## Comandos útiles
- `quarto render clases/` - renderiza todas las presentaciones
- `python scripts/ingest.py` - actualiza la base vectorial con documentos nuevos

## Reglas del proyecto
- Responder siempre en español formal, sin tuteos ni jerga informal.
- No estimar montos ni generar proyecciones financieras.
- Si la pregunta requiere interpretación legal, indicarlo explícitamente.
- Citar siempre la fuente documental usada en cada respuesta.
- Ante dudas sobre el alcance de una consulta, preguntar antes de responder.

## Contexto del dominio
- Los documentos de referencia están en `docs/normativa/`.
- La base vectorial se actualiza cada lunes con los circulares nuevos.
- El modelo de embeddings usado es `text-embedding-3-small` de OpenAI.

## Lo que NO debe hacer el agente
- Editar archivos en `docs/normativa/` sin confirmación explícita.
- Responder preguntas sobre casos individuales de afiliados.
- Generar código que acceda a sistemas de producción.
```

Qué logra este archivo

El agente llega al proyecto sabiendo qué hace, cómo debe comportarse, qué puede tocar y qué está prohibido. No hay que decírselo en cada conversación.

2. Skills

Una **skill** es una capacidad reutilizable y delimitada que un agente puede invocar.

Qué define una skill

- **nombre:** identificador de la capacidad
- **cuándo usarla:** condición de activación
- **instrucciones:** pasos o reglas de ejecución
- **salida esperada:** formato y contenido del output
- **herramientas o dependencias:** lo que necesita para operar

Por qué son contexto

Las skills inyectan instrucciones específicas solo cuando son relevantes. Evitan sobrecargar el contexto con todo al mismo tiempo.

Ejemplo de skill

```
---
name: clasificar_consulta_afp
description: Clasifica si una consulta previsional puede responderse automáticamente.
---

## Cuándo usarla
Cuando llega una consulta de afiliado antes de derivarla al flujo de respuesta.

## Instrucciones
1. Leer la consulta completa.
2. Clasificar en: informativa / operativa / sensible.
3. Si es sensible, marcar para revisión humana.

## Salida esperada
```json
{
 "tipo": "informativa | operativa | sensible",
 "razon": "explicación breve"
}
```

-----

## 3. Planes

Un plan es una descomposición estructurada de una tarea compleja en pasos ejecutables.

### Por qué es contexto

Le da al modelo (o al agente) una hoja de ruta explícita. En vez de improvisar cada paso, s

### Qué incluye un plan útil

- objetivo claro
- lista de pasos ordenados
- criterio de éxito por etapa
- formato esperado de cada output

### Cuándo conviene usarlo

Cuando la tarea tiene más de 3 pasos, cuando hay dependencias entre etapas o cuando quieres

-----

## Ejemplo de plan

```md

Plan: análisis de noticias financieras

Objetivo

Producir una tabla analítica a partir de noticias del día sobre Chile.

Pasos

1. Recolectar titulares del día (fuente: Google News)
2. Clasificar cada noticia por categoría (macro, mercados, regulación)
3. Estimar relevancia para Chile (1-5)
4. Detectar activos posiblemente impactados
5. Generar resumen ejecutivo por noticia
6. Consolidar en tabla final con todos los campos

Output esperado por etapa

- Paso 1: lista JSON de titulares
- Pasos 2-5: JSON por noticia con campos estructurados
- Paso 6: tabla Markdown lista para revisar

4. MCP — Model Context Protocol

Es un estándar abierto de Anthropic para conectar modelos con fuentes de datos y herramientas externas.

Qué hace

Permite que un modelo acceda en tiempo real a:

- bases de datos
- APIs
- sistemas de archivos
- servicios externos como calendarios, correo, Drive

Por qué es contexto

El modelo no tiene que recibir datos de antemano. Puede pedirlos cuando los necesita, a través de herramientas definidas por el servidor MCP.

MCP: la idea central

| Cliente (modelo) | Servidor MCP |
|--------------------------------|----------------------------|
| | |
| --- solicita recurso ---> | |
| | --- consulta base de datos |
| | --- llama a API |
| | --- lee archivo |
| <--- devuelve resultado -- | |
| | |
| usa el resultado como contexto | |
| para continuar la tarea | |

Diferencia con RAG

RAG recupera fragmentos de texto pre-indexados. MCP puede ejecutar acciones reales y devolver datos estructurados en tiempo real.

5. RAG — Retrieval-Augmented Generation

Ya lo vimos en detalle. Lo ponemos aquí en perspectiva.

Qué hace

Recupera fragmentos de documentos relevantes para una consulta y los inyecta en el contexto del modelo antes de generar la respuesta.

Cuándo es la opción correcta

- cuando el conocimiento está en documentos internos
- cuando la base documental cambia con cierta frecuencia
- cuando quieres respuestas con fuente explícita y trazable

Limitación clave

El modelo solo ve lo que el retrieval decidió traer. Un mal retrieval produce un mal contexto.

6. Memoria persistente

Es contexto que sobrevive entre sesiones.

Qué puede ser memoria

- preferencias del usuario
- decisiones pasadas
- resúmenes de conversaciones anteriores
- estado acumulado de un proceso de largo plazo

Cómo se implementa

- archivos de texto o Markdown leídos al inicio de cada sesión
- bases de datos vectoriales con histórico
- sistemas de notas del agente

Por qué importa

Sin memoria, cada sesión empieza desde cero. Con memoria bien diseñada, el modelo puede acumular criterio sobre el dominio.

7. Ejemplos — Few-shot

Mostrar ejemplos en el contexto es una de las formas más simples y efectivas de guiar el comportamiento del modelo.

Qué se hace

Se incluyen pares de entrada/salida dentro del prompt o del system message.

Por qué funciona

El modelo generaliza el patrón y lo aplica a entradas nuevas sin necesidad de instrucciones explícitas sobre el formato.

Cuándo usarlo

- cuando el formato de salida es crítico
- cuando hay lógica difícil de describir pero fácil de ejemplificar
- cuando los errores de formato tienen consecuencias (JSON mal formado, tabla mal estructurada)

Ejemplo de few-shot

Clasificá el tono de cada titular:

Titular: "Bolsa chilena sube 2% impulsada por cobre"

Tono: positivo

Titular: "Inflación alcanza máximo en 12 meses"

Tono: negativo

Titular: "Banco Central mantiene tasa sin cambios"

Tono: neutro

Ahora clasificá:

Titular: "Exportaciones de litio crecen por segundo trimestre consecutivo"

Tono:

Resumen: cuándo usar cada mecanismo

| Mecanismo | Cuándo conviene |
|----------------------------------|--|
| System prompt / CLAUDE.md | Reglas fijas del sistema o proyecto |
| Skills | Capacidades reutilizables que se activan por condición |
| Planes | Tareas complejas, multi-paso, que deben ser auditable |
| MCP | Acceso en tiempo real a datos o servicios externos |
| RAG | Base documental interna, respuestas con fuente |
| Memoria | Contexto acumulado entre sesiones |
| Few-shot | Guiar formato o lógica por ejemplo |

Parte III

Spec-driven development

¿Qué es spec-driven development?

Es una práctica de desarrollo donde antes de construir cualquier cosa se escribe una **especificación** detallada de lo que se quiere lograr.

En contextos de software, esa spec describe:

- qué debe hacer el sistema
- qué entradas y salidas tiene
- qué restricciones o reglas debe respetar
- qué casos límite deben manejarse

Por qué importa en IA

Cuando el agente o el modelo recibe una spec bien escrita como contexto, puede ejecutar tareas complejas con mucha más precisión y menos ambigüedad.

La spec como artefacto de contexto

Una especificación no es solo documentación.

Es un **artefacto de contexto de alta densidad**: concentra en un solo documento todo lo que el modelo necesita saber para ejecutar correctamente.

Qué incluye una buena spec

- objetivo del sistema o tarea
 - comportamiento esperado con ejemplos
 - formato de entradas y salidas
 - reglas de negocio y restricciones
 - criterio de éxito explícito
-

Spec-driven development con IA: el flujo

1. Escribir la spec
 - ↓ [artefacto: Markdown estructurado]
 2. Revisar y validar la spec con el equipo
 - ↓ [la spec es el contrato entre humanos y agente]
 3. Entregar la spec como contexto al agente
 - ↓ [el agente sabe qué hacer, cómo y hasta dónde]
 4. El agente ejecuta dentro del marco definido
 - ↓ [menor ambigüedad, menos correcciones]
 5. Los outputs se evalúan contra la spec
 - ↓ [la spec también es el criterio de evaluación]
-

Ejemplo: spec para el agente de RRHH

```
# Spec: asistente de consultas RRHH

## Objetivo
Responder preguntas frecuentes de empleados sobre vacaciones, beneficios
y reglamento interno, usando los documentos oficiales como única fuente.

## Entradas
- Pregunta en lenguaje natural del empleado
- Documentos de referencia: reglamento interno v2024, FAQ RRHH

## Salidas esperadas
- Respuesta en español formal, máximo 3 párrafos
- Referencia explícita al documento y sección usada
- Si la consulta es sensible: mensaje de derivación al ejecutivo

## Reglas
- No estimar ni proyectar montos (ej. liquidaciones, pensiones)
- No responder sobre casos individuales de otros empleados
- Si la información no está en los documentos, decirlo explícitamente

## Criterio de éxito
La respuesta es correcta si un ejecutivo de RRHH la aprobaría sin modificar.
```

Por qué la spec mejora todo el flujo

| Sin spec | Con spec |
|--|---|
| El agente interpreta lo que cree que se quiere | El agente opera dentro de un marco explícito |
| Los errores son difíciles de rastrear | Los errores se evalúan contra un criterio conocido |
| Cada cambio requiere reescribir el prompt | Se actualiza la spec y el comportamiento cambia consistentemente |
| Difícil de delegar o escalar | Cualquier persona puede leer la spec y entender qué hace el sistema |

Spec, plan y artefacto: las diferencias

| Concepto | Qué define | Cuándo se escribe |
|------------------|---|----------------------|
| Spec | Qué debe hacer el sistema, sus reglas y criterio de éxito | Antes de construir |
| Plan | Cómo se va a ejecutar una tarea concreta, paso a paso | Antes de ejecutar |
| Artefacto | El output concreto de una etapa del flujo | Durante la ejecución |

Relación entre los tres

La spec define el marco → el plan descompone la tarea → cada paso produce un artefacto → los artefactos se evalúan contra la spec.

Parte IV

Flujo multi-paso con artefactos estandarizados

La idea central

Cuando una tarea compleja se divide en pasos, cada paso debería producir un **artefacto** con formato estándar.

Ese artefacto tiene dos funciones:

1. **Output de la etapa:** resultado verificable y reutilizable
2. **Contexto de la etapa siguiente:** insumo explícito y controlado

Por qué el formato del artefacto importa

Si cada etapa produce texto libre:

- la etapa siguiente tiene que interpretar qué significa
- los errores se propagan sin que se detecten
- el flujo es difícil de auditar
- es imposible reutilizar resultados intermedios

Si cada etapa produce formato estandarizado (JSON, tabla, Markdown estructurado):

- la siguiente etapa puede consumirlo directamente
 - los errores se detectan en la etapa donde ocurrieron
 - el flujo es auditable paso a paso
 - los artefactos son reutilizables en otros flujos
-

Caso: evaluación de una iniciativa de IA

Vamos a seguir un ejemplo concreto.

Escenario: Una organización quiere evaluar si conviene implementar un chatbot interno para responder preguntas de RRHH.

Tarea: producir un análisis estructurado que permita tomar esa decisión.

¿Por qué este caso?

Porque requiere varios tipos de análisis, conecta con Decision Science y con lo visto en agentes, y el output importa para una decisión real.

Vista del flujo completo

- Etapas del flujo:
- Etapas 1: Definir el problema
↓ [Markdown: ficha de contexto]
 - Etapas 2: Mapear alternativas
↓ [Tabla: opciones comparadas]
 - Etapas 3: Identificar riesgos
↓ [JSON: riesgos estructurados]
 - Etapas 4: Definir criterios de evaluación
↓ [Tabla: criterios con peso]
 - Etapas 5: Generar recomendación
↓ [Markdown: recomendación ejecutiva]

Cada artefacto es el contexto explícito de la etapa siguiente.

Etapla 1: Definir el problema

Prompt de entrada

Ayúdame a estructurar el problema organizacional antes de evaluar si conviene un chatbot de RRHH. Genera una ficha de contexto en Markdown con estas secciones:

- Situación actual
 - Necesidad o dolencia
 - Decisión a tomar
 - Alcance preliminar
 - Supuestos relevantes
-

Artefacto 1: ficha de contexto (Markdown)

```
## Ficha de contexto: chatbot RRHH

### Situación actual
El área de RRHH recibe ~200 consultas mensuales por correo y teléfono, mayoritariamente sobre vacaciones, beneficios y reglamentos.

### Necesidad
Reducir tiempo de respuesta y liberar a los ejecutivos para tareas de mayor valor.

### Decisión a tomar
Determinar si conviene implementar un chatbot interno para consultas frecuentes.

### Alcance preliminar
Solo consultas informativas. No decisiones sobre liquidaciones ni casos disciplinarios.

### Supuestos
- El equipo puede mantener la base documental actualizada.
- Los empleados tienen acceso a Teams o Slack.
```

Por qué Markdown en esta etapa

- es legible para humanos y para modelos
- tiene estructura explícita con encabezados
- puede versionarse en Git como cualquier archivo
- es fácil de revisar y corregir antes de pasar a la siguiente etapa

Antes de seguir

Esta ficha se revisa y valida. Si algo está mal, se corrige aquí y no en etapas posteriores.

Etapa 2: Mapear alternativas

Prompt de entrada (incluye el artefacto anterior como contexto)

Dado este contexto:

[ficha de contexto aquí]

Lista las principales alternativas para resolver la necesidad.

Genera una tabla Markdown con columnas:

| Alternativa | Descripción | Costo estimado | Autonomía del sistema | Complejidad de implementación

Artefacto 2: tabla de alternativas (Markdown)

| Alternativa | Descripción | Costo estimado | Autonomía | Complejidad |
|-----------------------------------|--|----------------|-----------|-------------|
| Chatbot con FAQ estática | Bot simple con respuestas predefinidas | Bajo | Baja | Baja |
| Chatbot con LLM + RAG | Responde en lenguaje natural usando documentos | Medio | Media | Media |
| Formulario web + respuesta humana | Sin IA, proceso digitalizado | Muy bajo | Ninguna | Muy baja |
| Agente integrado a HRIS | Consulta datos reales del empleado | Alto | Alta | Alta |

Por qué tabla en esta etapa

- obliga a definir los mismos atributos para todas las opciones
 - facilita la comparación visual
 - puede importarse directamente en la etapa de evaluación
 - es fácil de presentar a tomadores de decisión
-

Etapa 3: Identificar riesgos

Prompt de entrada

Dado el contexto y las alternativas mapeadas:

```
[ficha de contexto]
[tabla de alternativas]
```

Identifica los principales riesgos de implementar un chatbot con LLM + RAG en RRHH.
Genera un JSON con esta estructura:

```
[
  {
    "riesgo": "nombre",
    "descripcion": "qué puede salir mal",
    "probabilidad": "alta | media | baja",
    "impacto": "alto | medio | bajo",
    "mitigacion": "qué se puede hacer"
  }
]
```

Artefacto 3: riesgos estructurados (JSON)

```
[
  {
    "riesgo": "Respuesta incorrecta sobre beneficios",
    "descripcion": "El modelo puede dar información desactualizada o ambigua",
    "probabilidad": "media",
    "impacto": "alto",
    "mitigacion": "Revisar documentación mensualmente y agregar disclaimer"
  },
  {
    "riesgo": "Sesgo de automatización",
    "descripcion": "Empleados confían en la respuesta sin verificar con RRHH",
    "probabilidad": "alta",
    "impacto": "medio",
    "mitigacion": "Incluir siempre referencia al documento fuente"
  },
  {
    "riesgo": "Consulta sensible mal manejada",
    "descripcion": "Caso disciplinario o médico recibe respuesta automática",
    "probabilidad": "baja",
    "impacto": "muy alto",
    "mitigacion": "Clasificar consultas y derivar a humano cuando corresponda"
  }
]
```

Por qué JSON en esta etapa

- tiene estructura fija que puede validarse automáticamente
 - cada campo tiene un valor definido y comparable
 - puede procesarse en la etapa siguiente (ordenar por impacto, filtrar por probabilidad)
 - puede persistirse en una base de datos o registro de riesgos
-

Etapa 4: Definir criterios de evaluación

Prompt de entrada

Dados los riesgos identificados y el contexto del problema:

```
[ficha de contexto]
[riesgos en JSON]
```

Define los criterios para evaluar si conviene avanzar con la alternativa "chatbot con LLM + RAG".

Genera una tabla Markdown con columnas:

```
| Criterio | Descripción | Peso (1-5) | Cómo medirlo |
```

Artefacto 4: criterios de evaluación (Markdown)

| Criterio | Descripción | Peso | Cómo medirlo |
|--------------------------------|--|------|---|
| Impacto en tiempo de respuesta | ¿Cuánto reduce el tiempo promedio de respuesta? | 5 | Comparar antes y después (días promedio) |
| Riesgo de error crítico | ¿Puede dar respuestas que generen daño real? | 5 | Revisión de casos límite con RRHH |
| Costo de mantención | ¿Cuánto esfuerzo requiere mantener actualizada la base documental? | 3 | Horas/mes estimadas |
| Adopción esperada | ¿Los empleados lo usarán? | 4 | Encuesta de intención previa al lanzamiento |
| Trazabilidad | ¿Se puede auditar qué respondió el sistema y con qué fuente? | 4 | Logs y referencias por respuesta |

Etapa 5: Recomendación ejecutiva

Prompt de entrada

Dado todo el análisis previo:

[ficha de contexto]
[tabla de alternativas]
[riesgos JSON]
[criterios de evaluación]

Genera una recomendación ejecutiva en Markdown con las secciones:

- Recomendación central (1 párrafo)
- Fundamentos (3 bullets)
- Condiciones mínimas para avanzar (lista)
- Riesgos a monitorear (lista)
- Próximos pasos sugeridos (lista numerada)

Artefacto 5: recomendación ejecutiva (Markdown)

```
## Recomendación: chatbot RRHH con LLM + RAG
```

```
### Recomendación central
```

```
Se recomienda avanzar con un piloto acotado para consultas informativas frecuentes, condicionado a que exista un proceso de mantención documental y una clasificación explícita de consultas que deben derivarse a humano.
```

```
### Fundamentos
```

- ```
- Potencial de reducción de carga operativa en consultas repetitivas
- Riesgos controlables si se delimita bien el alcance
- Costo de implementación razonable frente al valor esperado
```

```
Condiciones mínimas
```

- ```
- Base documental revisada y vigente antes del lanzamiento  
- Clasificador de consultas sensibles con derivación a ejecutivo  
- Disclaimer en cada respuesta con referencia al documento fuente
```

```
### Riesgos a monitorear
```

- ```
- Tasa de respuestas con información desactualizada
- Casos donde empleados tomaron decisiones basadas en el chatbot
```

```
Próximos pasos
```

- ```
1. Definir conjunto de preguntas frecuentes para el piloto  
2. Preparar base documental con RRHH  
3. Configurar chatbot con LLM + RAG en herramienta interna  
4. Correr piloto durante 4 semanas con grupo acotado  
5. Evaluar con criterios definidos y decidir si escalar
```

Qué produjo el flujo

Cinco artefactos encadenados, cada uno con un formato que:

- puede revisarse y corregirse de forma independiente
 - alimenta la etapa siguiente como contexto explícito
 - es reutilizable en otros flujos o análisis similares
 - es auditable: se puede rastrear de dónde viene cada conclusión
-

La ganancia real del flujo

| Sin flujo estructurado | Con flujo y artefactos |
|---|---|
| “Evalúa si conviene el chatbot” → texto libre | Cada etapa produce un formato verificable |
| Los errores se acumulan sin detectarse | Los errores se detectan en la etapa donde ocurren |
| El resultado es difícil de explicar | Hay trazabilidad completa del razonamiento |
| No se puede reutilizar | Cada artefacto puede alimentar otros análisis |
| El modelo tiene que inferir todo | El contexto explícito reduce la carga de inferencia |

Conexión con lo visto en el módulo

Este flujo usa todo lo que cubrimos:

- **Arquitectura LLM:** el modelo como capa de lenguaje, no como oráculo
 - **MLOps / CI/CD:** los artefactos son las unidades que se versionan y evalúan
 - **Decision Science:** estructura del problema antes de producir la recomendación
 - **Agentes y orquestación:** el flujo puede automatizarse en n8n o herramienta similar
 - **Contexto:** cada artefacto previo se inyecta como contexto de la etapa siguiente
-

Principio unificador

El contexto bien construido es la diferencia entre un modelo que responde y un sistema que resuelve.

Un modelo sin contexto adivina.

Un modelo con contexto estructurado, generado etapa a etapa y en formatos estandarizados, razona sobre información explícita y produce salidas confiables.

Cierre

Lo que quedó del módulo

- el modelo es una pieza, no toda la solución
 - el contexto es el insumo más importante
 - hay formas distintas de construir contexto según el tipo de problema
 - los flujos multi-paso producen artefactos que encadenan el razonamiento
 - cada etapa con formato estandarizado hace el sistema más trazable y confiable
-

Mensaje final

No se trata de usar IA porque es la tendencia.

Se trata de entender cómo darle contexto útil, cómo dividir tareas en pasos manejables y cómo producir outputs que alimenten decisiones reales.

El modelo es potente por defecto.

El contexto es lo que hace que esa potencia sea tuya.

Referencias sugeridas

- Anthropic, “Model Context Protocol”. <https://docs.anthropic.com/en/docs/mcp>
- OpenAI, “Prompt engineering”. <https://platform.openai.com/docs/guides/prompt-engineering>
- Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <https://arxiv.org/abs/2005.11401>
- Materiales del módulo: arquitectura LLMs, MLOps, Decision Science, agentes y caso AFP